RepoRAG

Project under the Information Retrieval Course

Anshul Rath
Devansh Agarwal

Group 9

Introduction

- A large portion of developer time goes into **understanding codebases** rather than building features.
- Working with new repositories requires **clear and intuitive explanations of logic.**
- Traditional search methods rely on keywords and often miss the **actual intent and structure.**
- We aim to build a system that can:
  - Find the **right pieces of code.**
  - Explain **how and why they work.**

Pipeline Overview

Code Understanding

- Source files are parsed into structured representations using AST
- Each function is embedded for semantic search
- A dependency graph captures relationships across the codebase

Query Handling

- User queries are refined using a lightweight language model
- Retrieval combines both semantic and keyword-based signals
- Results are ranked using reciprocal rank fusion (RRF)

Response Generation

- Relevant code regions are expanded using graph context
- Structured input is passed to the LLM
- Final output explains the logic in a readable way

Tech Stack

Application Layer

- Frontend built using **ReactJS**
- Backend services powered by **FastAPI**

Core System

- **Tree-Sitter** is utilized for the task of Code parsing
- Code embeddings generated using **jina embeddings**
- LLM support via **Groq API** and **Qwen, Gemma(local)**
- Embeddings stored in a lightweight **JSON format**

Language Coverage

- C/C++, Java, Python, JavaScript, Jsx, Ts

Ground Truth Construction

How we build ground truth

- Functions extracted from a repository chosen for evaluation (using the indexer pipeline)
- Queries generated using appropriate LLM API calls and structured prompt.

Single-Hop Queries

- Answered using **one function only**
- Covers:
  - Direct queries (explicit names)
  - Meaning-based queries (intent / logic)
  - Structural role in the codebase
- Each query maps to a **single ground truth**

Multi-Hop Queries

- Require reasoning across **multiple functions**
- Built using local **dependency neighborhoods**
- Focus on only relevant internal links
- Each query maps to a **set of functions**

Retriever Evaluation Metrics

Ranking Quality

- **MRR:** Assgins higher scores to systems that rank relevant results higher.
- **nDCG@k:** Measures the quality of the ranking by considering both relevance and position of results. Higher nDCG indicates better ranking.

Coverage (Context Retrieval)

- **Recall@k:** Ability of the system to retrieve all relevant items within the top k results.

Evaluation Depth

- Metrics computed at:
  - Top 3
  - Top 5
  - Top 7

Focus: balancing **accuracy** and **ranking quality**

Results — Single Hop

Method	MRR	R@3	R@5	R@7
BM25	0.45	0.56	0.65	0.70
BM25 + Dep	0.44	0.55	0.66	0.70
BM25 + Dense	**0.67**	**0.76**	**0.84**	0.87
BM25 + Dense + Dep	0.44	0.60	0.80	**0.88**

- Dense retrieval significantly improves performance
- Hybrid approach achieves the best overall ranking
- Dependency information helps at higher recall levels

Results — Multi Hop

Method	R@3	nDCG@3	R@5	nDCG@5	R@7	nDCG@7
BM25	0.54	0.58	0.65	0.63	0.73	0.66
BM25 + Dep	0.68	0.68	0.78	0.74	0.82	0.76
BM25 + Dense	0.76	0.77	0.82	0.80	0.85	0.82
BM25 + Dense + Dep	**0.71**	0.68	**0.85**	0.75	**0.91**	0.78

- Multi-hop queries benefit from **dependency context**
- Combining dense + structural signals from the graph gives best coverage
- Strong gains observed at deeper retrieval levels

Generator Evaluation

Evaluation Setup

- Responses evaluated using an **LLM-as-a-judge** (Gemma 4 31B)
- Focus on:
  - Correctness
  - Completeness (Does the answer address the complete query)
  - Clarity
  - Faithfulness to retrieved context (Answer grounded in the retieved context)
- Score:
  - 0 = Poor - Does not meet the criteria
  - 1 = Partial — Partially meets the criteria
  - 2 = Good — Fully meets the criterion
- Evaluated over 5 handpicked queries from ground truth
- Larger models produce more consistent explanations
- Local models offer a strong trade-off between cost and quality

Generator Evaluation

Llama 3.3 70B Versatile v/s gemma3:4b

Single Queries

Model	Faithfulness	Correctness	Completeness	Clarity
Llama 3.3 70B Versatile (API)	2	2	2	2
gemma3:4b (Local)	1.4	0.6	1.0	1.8

Multi-Hop

Model	Faithfulness	Correctness	Completeness	Clarity
Llama 3.3 70B Versatile (API)	2	2	2	2
gemma3:4b (Local)	1.2	1.0	0.4	1.8

Generator Evaluation

Llama 3.3 70B Versatile v/s qwen2.5-coder:7b

Single Queries

Model	Faithfulness	Correctness	Completeness	Clarity
Llama 3.3 70B Versatile (API)	2	1.8	1.8	1.8
qwen2.5-coder:7b (Local)	2.0	2.0	1.8	2.0

Multi-Hop

Model	Faithfulness	Correctness	Completeness	Clarity
Llama 3.3 70B Versatile (API)	2	2	1.6	1.8
qwen2.5-coder:7b (Local)	1.2	1.0	1.4	2.0

Generator Evaluation

qwen2.5-coder:7b v/s gemma3:4b

Single Queries

Model	Faithfulness	Correctness	Completeness	Clarity
qwen2.5-coder:7b (Local)	2	2	2	2
gemma3:4b (Local)	1.2	1.0	0.8	1.6

Multi-Hop

Model	Faithfulness	Correctness	Completeness	Clarity
qwen2.5-coder:7b (Local)	1.2	1.2	1.8	2.0
gemma3:4b (Local)	1.2	1.0	0.8	1.6

Comparison with Agentic RAG

- Relevance: Context contains the code related to the query
- Completeness: Captures all the code snippets related to the query
- Precision: Retrieved context focussed with minimal noise

Context Scores

System	Relevance	Completeness	Precision
Agentic RAG	0.6	0.4	0.6
RepoRAG (Ours)	**2.0**	**2.0**	**1.6**

Answer Scores

System	Faithfulness	Correctness	Completeness	Clarity
Agentic RAG	1.6	0.4	0.2	2.0
RepoRAG (Ours)	**2.0**	**2.0**	**2.0**	**2.0**

Conclusion

- Designed a structured pipeline for understanding code repositories
- Combined semantic retrieval with dependency-aware reasoning
- Achieved strong performance across both single-hop and multi-hop queries
- Maintained a balance between **accuracy, efficiency, and simplicity**

Thank You